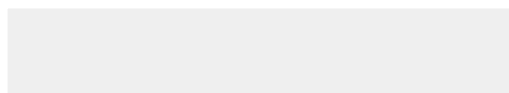




Advanced Time Series - Computer Exercises 1

AUTHORS



GROUP: ASSIGNMENT 1 5

September 18, 2026

Contents

1	Part 1 - Simulation of Different Non-Linear Models	1
1.1	SETAR(2,1,1)	1
1.2	IGAR(2,1)	2
1.3	MMAR(2,1)	3
1.4	Comparison	4
2	Part 2 - Compare Theoretical and Estimated Conditional Mean	6
3	Part 3 - Cumulative Conditional Mean	8
4	Part 4 - Conditional Parametric Model Estimation	10
5	Part 5 - Identifying non-linearities with use of LDF	12
6	Appendix A	14

1 Part 1 - Simulation of Different Non-Linear Models

1.1 SETAR(2,1,1)

We start by looking at a SETAR(2,1,1) model. Which can be written as,

$$X_t = \begin{cases} a_{10} + a_{11}X_{t-1} + \epsilon_t & \text{if } X_{t-1} \in R_1 \\ a_{20} + a_{21}X_{t-1} + \epsilon_t & \text{if } X_{t-1} \in R_2 \end{cases}$$

We can see, that this is a SETAR(l, d, k) model with $l = 2$, as we have two regimes. We have $d = 1$, because it is the first lag X_{t-1} that determines the switch between regimes and finally $k = 1$ because we only sum up to 1 in the term $\sum_{i=1}^k a_i X_{t-i}$

In each regime, the time series behaves like a linear AR model, but with different coefficients. Unlike regular AR models, the SETAR model shows non-linear dynamics, as regime switching causes sharp behavior changes that a single linear model cannot capture. What makes the SETAR model unique is that it's a self-exciting threshold AR-model, where regime changes depend on the lagged dependent variable, X_{t-1} in the SETAR(2,1,1) case.

We simulate three cases in R with different parameters. In all cases, regimes are defined by the condition $X_{t-1} \leq 0$ and $X_{t-1} > 0$ determining whether values fall into the first or second regime. The plots are shown in Figure 1

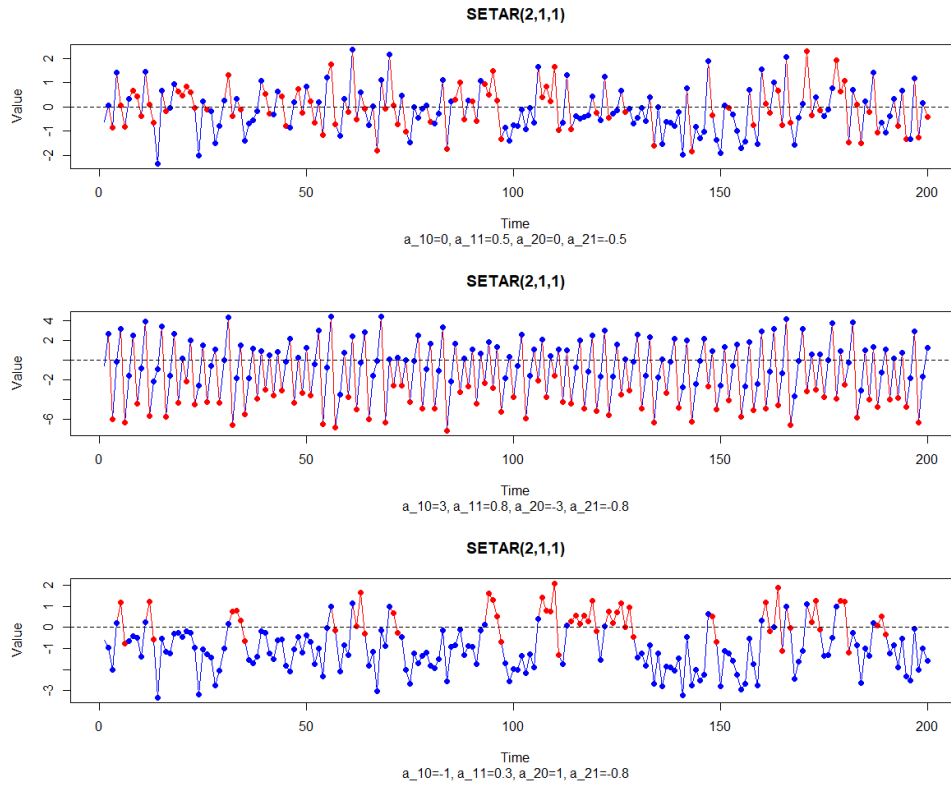


Figure 1: Plot of SETAR(2,1,1) models with different parameters. Points are colored blue when in the first regime, while points in the second regime are colored red.

We observe in all three cases, a switch to the first regime occurs when the previous X -value is positive,

and to the second regime when it's negative, making regime switches relatively easy to predict.

In the top plot of Figure 1, both intercepts are $a_{10} = a_{20} = 0$ causing the regimes to oscillate around zero. The slopes have equal magnitude but opposite signs, creating symmetrical behavior. In the first regime (blue), the series trends upward until a switch, while in the second regime (red), it trends downward.

In the middle plot, the intercepts $a_{10} = 3$ and $a_{20} = -3$, cause the regimes to oscillate around different means. The positive intercept in the first regime (blue) pulls the series sharply upward, while the negative intercept in the second regime (red) pulls it sharply downward. The frequent sharp shifts between regimes are due to these contrasting intercepts exerting strong upward or downward forces.

In the bottom plot, the regimes have different slope magnitudes $a_{11} = 0.3$ and $a_{21} = -0.8$. The first regime (blue) has a smaller slope magnitude, causing X -values to rise more slowly compared to how fast they fall in the second regime (red). As a result, the series stays in the first regime longer before switching.

1.2 IGAR(2,1)

Now we look at a IGAR(2,1) model, which can be written as,

$$X_t = \begin{cases} a_{10} + a_{11}X_{t-1} + \epsilon_t & \text{with prob } p \\ a_{20} + a_{21}X_{t-1} + \epsilon_t & \text{with prob } p - 1 \end{cases}$$

We can see, that this is a IGAR(l,k) model with $l = 2$, as we have two regimes and $k = 1$ because we only sum up to 1 in the term $\sum_{i=1}^k a_i X_{t-i}$

Like the SETAR model, the IGAR model is a piecewise AR model with non-linear dynamics, as regime switches result in a behaviour change that a single linear model can't capture. What makes the IGAR model unique is that in the IGAR model, regime switches occur based on a probability. This makes the trend of the model less predictable. Otherwise, the behavior within each regime and the parameter effects are similar to the SETAR model.

We simulate three IGAR(2,1) models with constant parameters, varying only the regime-switching probabilities to examine their impact on the model's behaviour.

Figure 2 shows that a high probability of switching to the first regime (blue) results in more data points from that regime. If the probability is low, more data points come from the second regime (red). When the probability is 0.5, the data points are roughly balanced between both regimes.

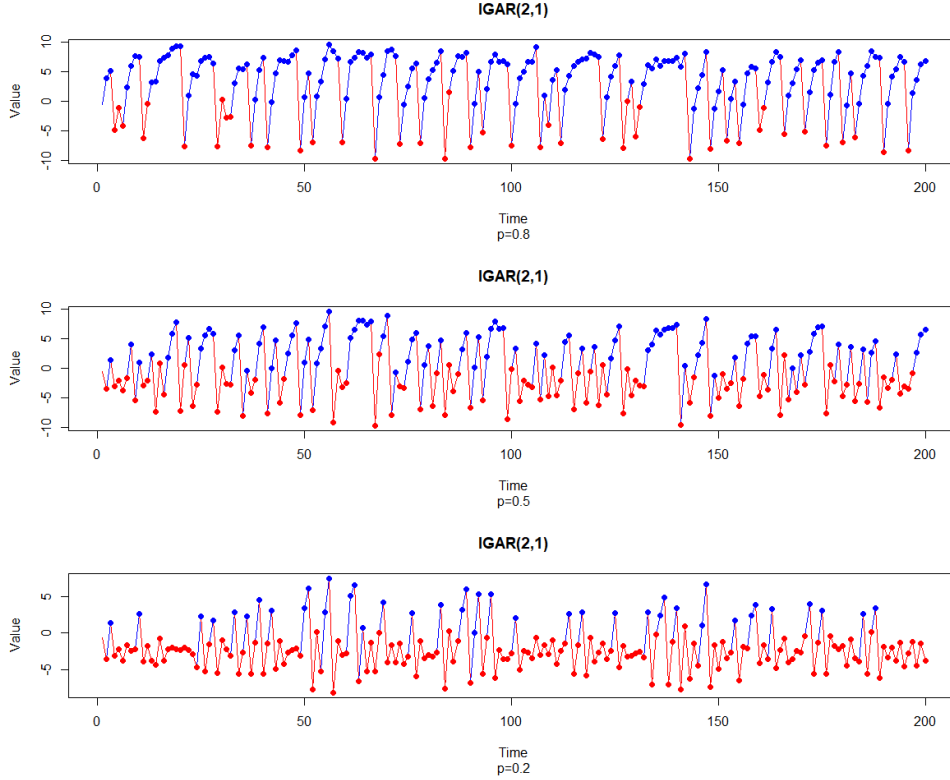


Figure 2: Plot of IGAR(2,1) models with the same parameters, but different probabilities. Data points from the first regime are colored **blue**, while data points from the second regime are colored **red**.

1.3 MMAR(2,1)

Now we look at a MMAR(2,1) model. The model can be written as,

$$X_t = \begin{cases} a_{10} + a_{11}X_{t-1} + \epsilon_t \\ a_{20} + a_{21}X_{t-1} + \epsilon_t \end{cases}$$

The transition between the different regimes is governed by the transition matrix

$$P = \begin{pmatrix} p_1 & 1 - p_1 \\ 1 - p_2 & p_2 \end{pmatrix}$$

Like the other models, the MMAR is a piecewise AR model with non-linear dynamics, where regime switching changes behavior that a single linear model can't capture. What sets the MMAR apart is that transitions between regimes are governed by a Markov chain, allowing different probabilities for staying in the same regime versus switching

We simulate three MMAR models with the same parameters, varying only the transition probabilities. For simplicity we will have $p_1 = p_2$ in all cases. The simulations are plotted in Figure 3.

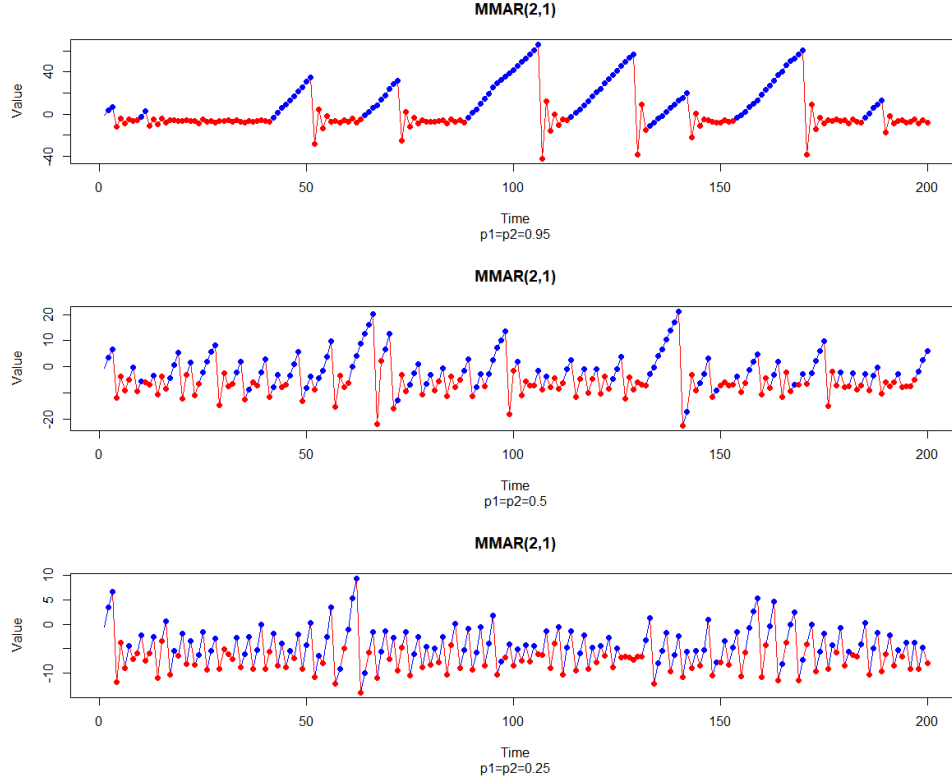


Figure 3: Plot of MMAR(2,1) models with the same parameters, but different transition probabilities. Data points from the first regime are colored blue, while data points from the second regime are colored red.

Looking at the top plot in Figure 3 we can see that as the probability of staying in the same regime is greater than switching regimes, we have longer periods without regime switch. The opposite can be observed in the bottom plot, where the probability of switching regimes is higher than staying in the same regime - resulting in very frequent switch of regimes.

The middle plot is the case, where the probability of staying in a regime and switching regime is the same. In this case, the MMAR model(2,1) is the same as an IGAR(2,1) model, where the probabilities are 0.5 for ending in either regime.

1.4 Comparison

Finally we compare the three models, using the same parameters for all three models. The SETAR(2,1,1) switches regimes at $X_{t-1} \leq 0$ and $X_{t-1} > 0$. The IGAR(2,1) model has a 0.7 probability of being in the first regime. The MMAR(2,1) model has transition probabilities $p_1 = p_2 = 0.7$. The results are shown in Figure 4

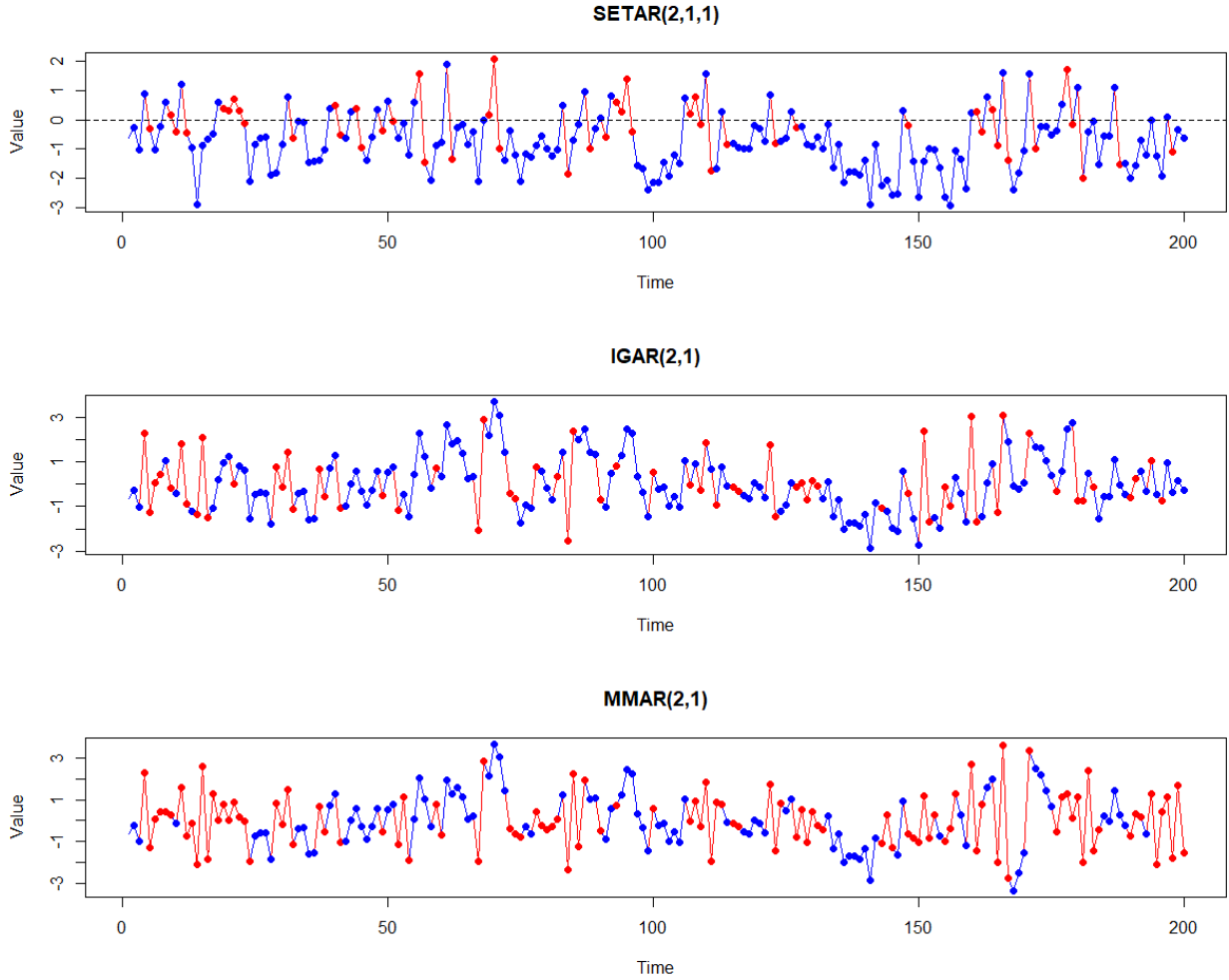


Figure 4: Plot of SETAR(2,1,1), IGAR(2,1) and MMAR(2,1) models with the same parameters. The first regime is colored **blue**, while the second regime is colored **red**.

From the plot we can see, that the trend of the SETAR model is much more predictable, as we simply need to look at the X_{t-1} value to determine if a switch happens. In contrast, the IGAR and MMAR models have stochastic regime switches, making them less predictable. Comparing the IGAR and MMAR models, we see that the IGAR model clearly tends toward the first regime due to the higher probability, while the MMAR model does not show this tendency, as it has longer periods of staying in the same regime. The key differences we notice between the three models are the following:

- **SETAR**: The regime changes are somewhat predictable, as they are based on previous values.
- **IGAR**: Random regime switches based on probability, independent of previous values, making the switches and thereby also the trend less predictable.
- **MMAR**: The probability of switching regime is dependent on the regime the series is currently in. This means it is less random than IGAR, but as it is based on transition probabilities of a Markov chain, it is less predictable than the SETAR.

2 Part 2 - Compare Theoretical and Estimated Conditional Mean

In this section we will consider the following $SETAR(2, 1, 1)$ model

$$X_t = \begin{cases} -0.8X_{t-1} + \epsilon_t & X_{t-1} \geq 0 \\ 0.8X_{t-1} + 1.5 + \epsilon_t & X_{t-1} < 0 \end{cases}$$

Since $\epsilon \sim N(0, \sigma^2)$ the theoretical conditional mean becomes

$$M(x) = E\{X_{t+1}|X_t = x\} = \begin{cases} -0.8x & x \geq 0 \\ 0.8x + 1.5 & x < 0 \end{cases}$$

Next we simulate 1000 values from the considered $SETAR$ model. This is done by first generating 1000 residuals from a normal distribution with $\mu = 0$ and $\sigma = 1$, then setting $X_1 = \epsilon_1$ and from there on computing X_{t+1} using the definition of the model.

To estimate the conditional mean of the simulated data we use the kernel estimator, which is a local regression estimator. We try this with 2 different kernels, the Epanechnikov kernel and the Tricube kernel. The Epanechnikov kernel with bandwidth h is given by

$$k^{Epa}(uh^{-1}) = \frac{3}{4} \left(1 - \frac{u^2}{h^2}\right) 1_{\{|u| \leq h\}}.$$

When using the Tricube kernel, you choose a parameter $\bar{h}$, which controls the proportion of the total dataset that is used to fit each target point. For example, if $\bar{h} = 0.5$, then 50% of the data closest to the target point are included in the fit. The weight assigned to each selected data point is given by

$$w(X_i) = \left(1 - \left(\frac{|dist|}{|maxdist|}\right)^3\right)^3$$

where $dist$ is the distance between the data point X_i and the target point and $maxdist$ is the maximum distance from the target point to any of the selected data points. In this way, the weight decreases as the distance from the target point increases.

Since the model only has one regressor, i.e $q = 1$, the estimated conditional mean, $\hat{M}(x)$, using the Epanechnikov kernel, can be found in the following way:

$$\hat{M}(x) = E\{X_{t+1}|X_t = x\} = \frac{\frac{1}{N} \sum_{s=2}^N X_s k\{h^{-1}(x - X_{s-1})\}}{\frac{1}{N} \sum_{s=2}^N k\{h^{-1}(x - X_{s-1})\}} \quad (1)$$

When using Tricube kernel for estimation, we have used the `loess()` function in R. The theoretical and estimated conditional means are shown in Figure 5 together with the simulated data points.

When using the Epanechnikov kernel, a small bandwidth leads to a fitted curve that is very non-smooth. This behavior is seen for $h = 0.2$ and $h = 0.5$. While these curves appear to capture the discontinuity that occurs at 0.0, they tend to fit the noise present in the continuous sections of the model rather than accurately reflecting the underlying trend.

On the other hand, using a large bandwidth produces a smoother fitted curve, it tends to overestimate the underlying trend. This effect is particularly seen for $h = 1.5$. While this larger bandwidth results in a more stable curve, especially at the edges (see red curve in left plot), it does not capture

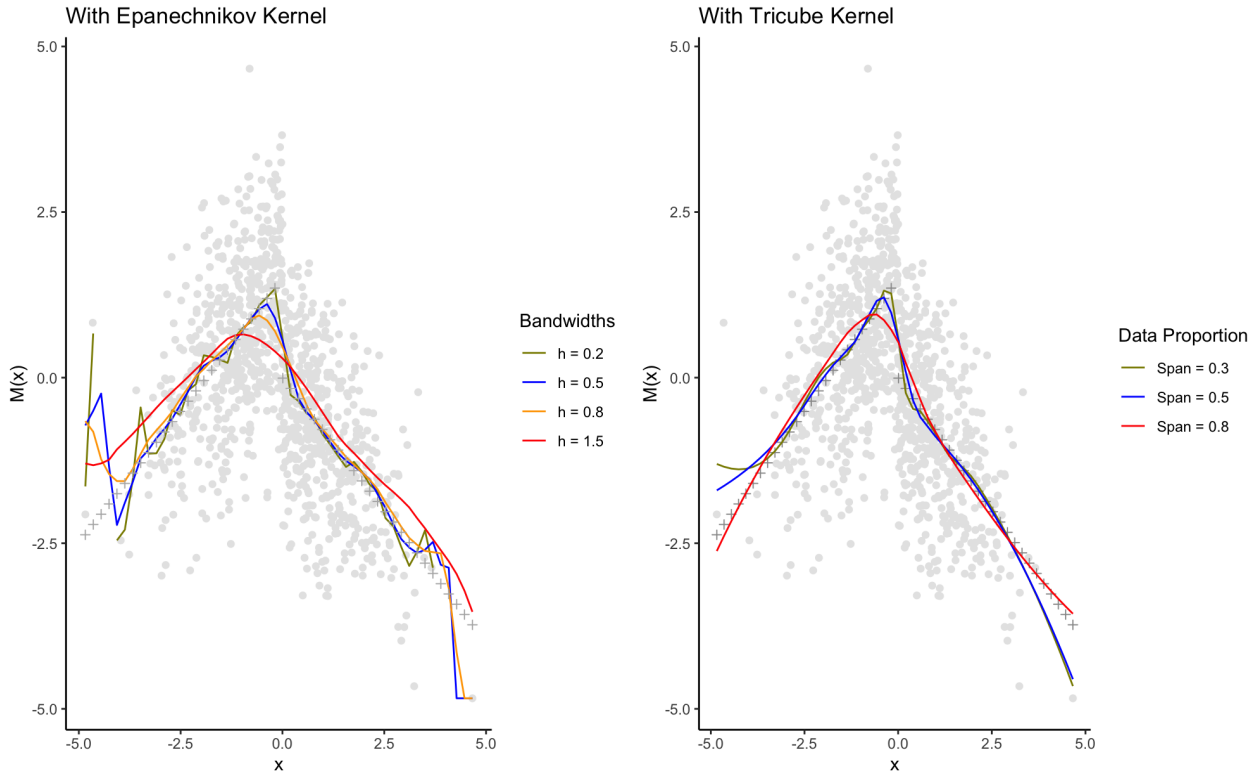


Figure 5: The figure displays the conditional mean together with the simulated data from a $SETAR(2,1,1)$ model. The theoretical conditional mean is shown with '+'. The estimation using the Epanechnikov kernel (left) is conducted with four different bandwidths, while the estimation using the tricube (right) kernel is performed with three different spans.

detail on local variations, such as discontinuities or sharp changes. The stability at the edges for large bandwidths compared to small is expected because there are fewer data points in that area and a larger bandwidth incorporates more data points in the estimation, leading to greater averaging and smoothing. In contrast, with a smaller bandwidth, fewer nearby points are used, making it difficult to provide stable estimates at the edges, where there are fewer data points.

Among the different bandwidths considered for the Epanechnikov kernel the yellow curve with $h = 0.8$ appears to be the one that best captures the underlying trend of the model without over fitting the noise.

The Tricube kernel takes care of the instability at the edges better than the Epanechnikov kernel since it uses the same amount of data points for every estimate and assigns weights based on relative distance. It also seems to provide an overall more stable fit than Epanechnikov.

As when using a large bandwidth in the Epanechnikov kernel a large span with the Tricube kernel provides a good estimate at the edges but does not capture the discontinuity at 0.0 as effectively as the smaller span do. A smaller span better preserves sharp changes or discontinuities, but lacks performance at the edges.

3 Part 3 - Cumulative Conditional Mean

We use the same SETAR(2,1,1) model as in part 2, and calculate the estimated cumulative conditional mean(CCM) using the equations in (3.6) and (3.8):

$$\hat{\Lambda}(\cdot) \int_a^\cdot \hat{\lambda}(x)dx, \quad \hat{\lambda} = \frac{(nh_n)^{-1} \sum_{t=1}^n X_t I\{X_{t-1} \in I_x\}}{(nh_n)^{-1} \sum_{t=1}^N I\{X_{t-1} \in I_x\}}$$

From equation (3.4) we know that the theoretical cumulative conditional mean(CCM) is calculated as,

$$\Lambda(\cdot) \int_a^\cdot \lambda(x)dx$$

where $\lambda(x)$ is the theoretical conditional mean defined just like in part 2,

$$\lambda(x) = E\{X_{t+1}|X_t = x\} = \begin{cases} -0.8x & x \geq 0 \\ 0.8x + 1.5 & x < 0 \end{cases}$$

The computation of the cumulative conditional means(CCM) is made in R, and where we have taken large inspiration for the R script `cumulativeMeans.R`. For computation of the estimated CCM, we require at least 5 observations per bin to ensure statistical reliability. The bin boundaries are defined over the interval $[-3.3, 3.3]$.

It's also important to note that the estimated conditional mean for each bin is plotted at the midpoint of the interval, as averaging over the bin makes it logical to represent the value there.

The results are shown in Figure 6 below,

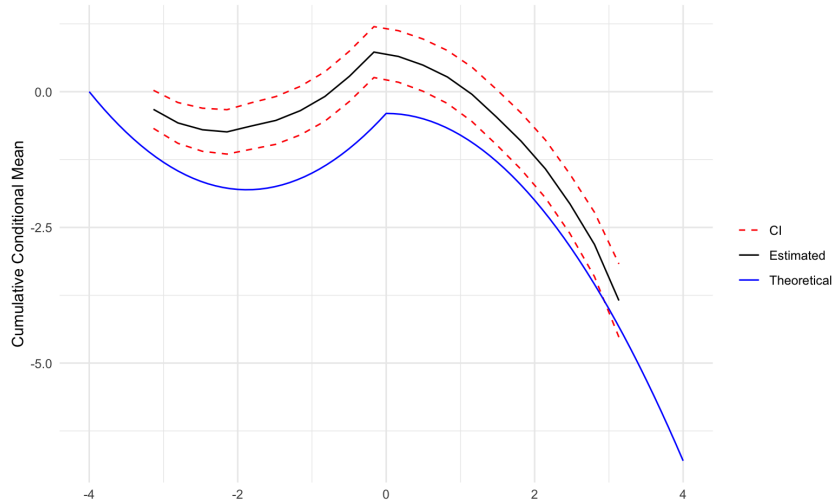


Figure 6: Theoretical and estimated cumulative conditional mean together with confidence bands

The theoretical CCM initially decreases until about -2, where it starts increasing, peaking around 0 before decreasing again. This behaviour is expected as we saw in part 2 Figure 5, that the conditional mean is negative until around -2 where it turns positive, and then negative again at 0.

Since the CCM is the summation of conditional means, its behavior reflects the sign of the conditional - adding negative values when the conditional mean is negative, and positive values when the conditional mean is positive. Hence, the theoretical CCM behavior shown in Figure 6 is expected.

We see that the estimated CCM behaves similarly but shifts slightly before points -2 and 0. This may be due to binning during histogram regression, which introduces approximation errors, along with potential estimation bias. These factors can cause the estimated mean to react earlier or later than the theoretical one.

As previously mentioned we plotted the estimated λ 's in the middle of each interval. If we had plotted them at the end of each interval instead of the middle, the estimated CCM would likely align more closely with the theoretical one, as this would cause a shift to the right.

Figure 6 shows that the estimated CCM consistently overestimates the theoretical CCM. However, as we move along the axis, that the estimated CCM gets closer to the theoretical one, and finally crossing the confidence bands. The slopes suggest that the estimated CCM could possibly converge towards the theoretical one. To further explore the asymptotic behavior and check if the estimated CCM actually converges to the theoretical, we could either reduce the minimum sample requirement per bin (which may increase noise) or simulate more data, which is the preferred approach.

One reason for the overestimation is that the empirical distribution is subject to sample noise and small-sample bias, which influences the estimation procedure. If early data points fall into the negative regime $X_{t-1} \leq 0$ the constant term 1.5 causes upward bias in X_t leading to larger early estimates. The combination of small-sample variability and fewer points in early bins contributes to this bias.

The cumulative mean adds together the means over intervals. If early intervals overestimate the theoretical mean, the bias will remain, but shrinks as later data better reflects the model dynamics. The Law of Large Numbers explains why this bias decreases, leading the empirical mean to converge to the theoretical mean as more data is collected.

4 Part 4 - Conditional Parametric Model Estimation

In this section, we will estimate the heat loss coefficient, U_a . We have been given, that the heat load of buildings can be described by the following model

$$\Phi_t = U_a(T_t^i - T_t^e) + \epsilon_t,$$

where U_a depends non-linearly on the wind speed, i.e $U_a = U_a(W_t)$. Considering U_a as a non-parametric function (i.e., assuming that $U_a(W_t)$ does not follow a specific parametric distribution/form), we will estimate this function by fitting a conditional parametric model.

We have been given a time series consisting of Φ_t , T_t^i , T_t^e and W_t , from which we can compute a time series for $U_a(W_t)$ in the following way:

$$U_a(W_t) = \frac{\Phi_t}{T_t^i - T_t^e}$$

Next, we can consider the conditional parametric model:

$$U_a(W_t) = \theta(W_t) + \epsilon_t$$

where the parameters θ can be estimated locally by solving:

$$\arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N w_i(x) (U_a(W_i) - \theta(x)). \quad (2)$$

The fact that the parameters are estimated locally means that equation 2 must be solved for every given wind speed. Note that equation 2 is a weighted least squares problem, and to handle the weights for a given wind speed, we will use the Epanechnikov kernel. We chose to have a quadratic assumption of $\theta(x)$ to better capture potential local curvature.

In Figure 7, a plot of the estimation of $U_a(W_t)$ is provided, using three different bandwidths.

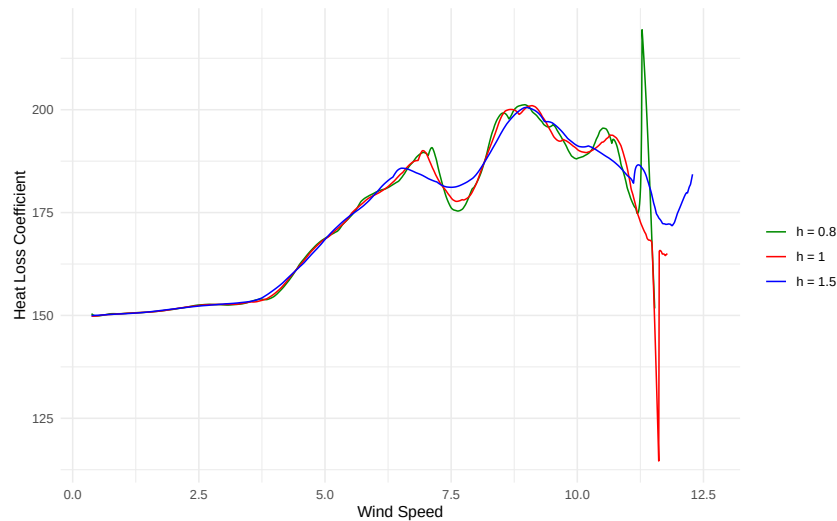


Figure 7: Estimation of the heat loss coefficient, using the Epanechnikov kernel with 3 different bandwidths.

At first glance, the figure indicates that the heat loss coefficient generally shows an increasing trend with rising wind speed. This is also intuitive, as heat loss from a building is most likely to rise with increasing wind speed due to more air movement.

The figure also clearly illustrates that the heat loss coefficient is non-linearly dependent on wind speed. The coefficient appears to remain relatively constant or have a slight positive relationship for wind speeds ranging from 0.0 to approximately 4. In the range of 4 to around 7, the coefficient displays a more positive linear relationship with wind speed. Beyond this point, the dependency becomes more variable.

For bandwidths of $h = 0.8$ and $h = 1$, noticeable up and down spikes occur for large W_t , and the estimation stops before reaching the maximum wind speed recorded in the dataset. This can be explained by that fact that there are only two data points with wind speed higher than 11. Consequently, with a small bandwidth, the weights assigned to each observation drop to zero, leaving the weighted least squares problem unsolvable. This limitation makes it challenging to discern any meaningful relationship between the heat loss coefficient and wind speed at higher speeds.

5 Part 5 - Identifying non-linearities with use of LDF

When modelling the data using an ARMA model, we plot the ACF and PACF of the series in order to determine the initial order of the model.

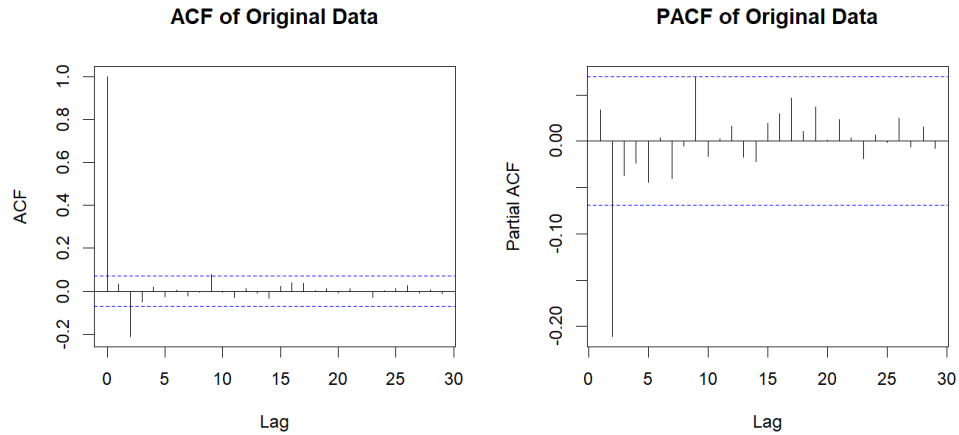


Figure 8: ACF and PACF of the data.

From Figure 8 we see that for the ACF we have a significant lag at 2, while all lags afterwards are within the confidence intervals. For the PACF we have a lag at 1. We therefore start by fitting an ARMA(1,2). Plotting ACF and PACF of the residuals and performing a Liung-Box statistical test we get the following plots,

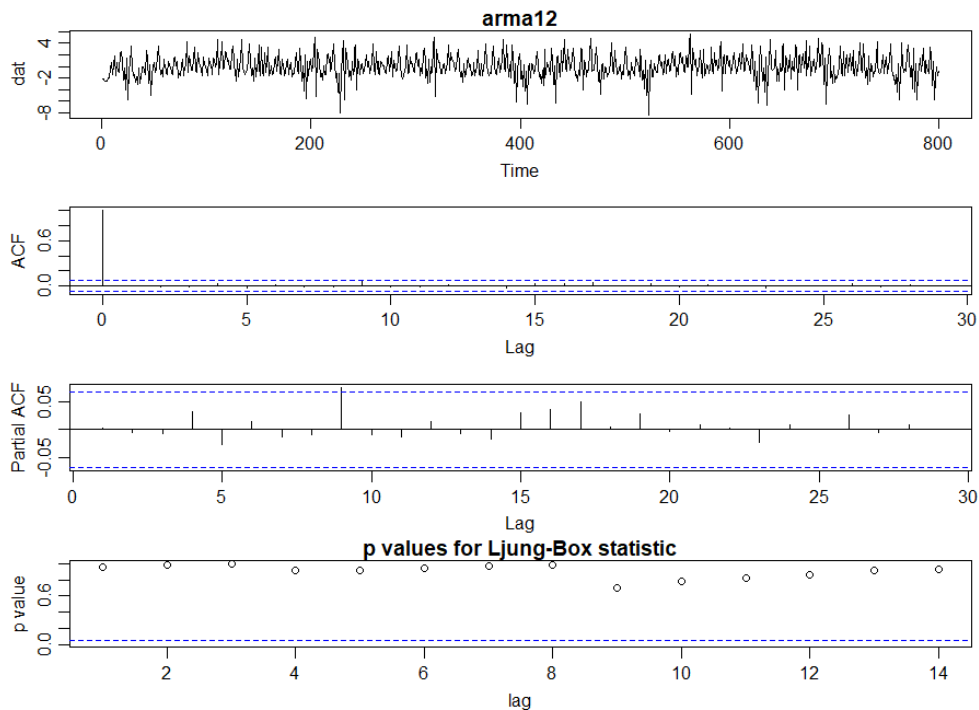


Figure 9: ACF and PACF and of residual of the ARMA(1,2) model as well as a Liung-Box test.

Based on Figure 9 the residuals seem to appear as white noise and therefore we do not increase the model order any further. We do note, that when plotting the residuals for models of a lower order than ARMA(1,2) we do not get white noise residuals.

As we are now reasonably satisfied with the ARMA(1,2) model, we compute the Lag Dependent Function (LDF) of the residuals using the given function provided in the R-scripts `ldf.R` and `leaveOneOut.R`. The result is the LDF presented in Figure 10.

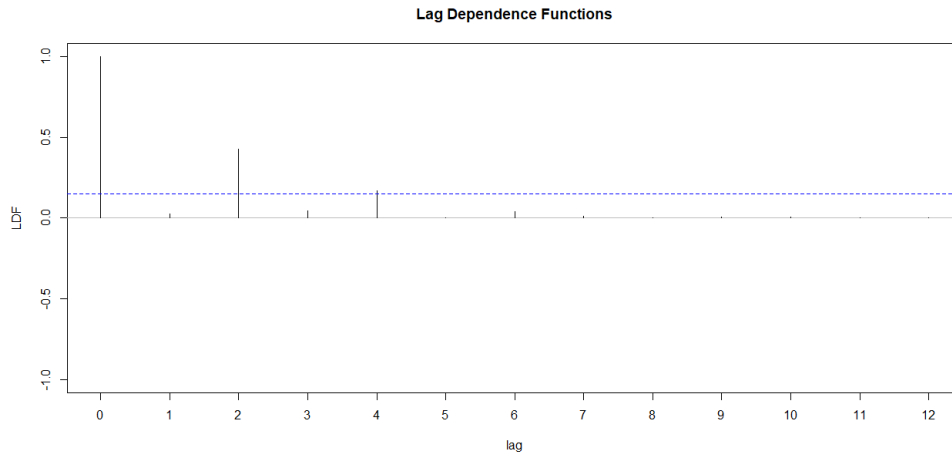


Figure 10: LDF of the residuals.

Since the LDF is designed to detect dependencies in time series, we would expect the LDF of the residuals to show no significant lag, for $\text{lag} > 0$, if the model structure were correct. However, the LDF of the residuals shows a significant lag in lag 2, suggesting that the ARMA(1,2) model fails to account for some structure in the data. Specifically, this significant lag 2 implies a relationship between the residuals at time t and $t - 2$ that is not captured by the current model.

Figure 11, which shows ϵ_{t-2} plotted against ϵ_t , supports this detected relationship. The plot suggests a nonlinear dependence between the residuals at lag 2 and the current residuals. This nonlinearity is not captured by the linear structure of the ARMA model.

Therefore, a better proposed model structure could be a Nonlinear Moving Average (NLMA(2)) model. The residual plot gives a hint of a piecewise linear relationship between ϵ_t and ϵ_{t-2} , with the possibility of three or four distinct slopes, suggesting that the relationship may vary across different regimes. In conclusion, the significant lag at 2 in the LDF and the nonlinear pattern in the residual plot both point towards the need for a nonlinear model, such as NLMA(2).

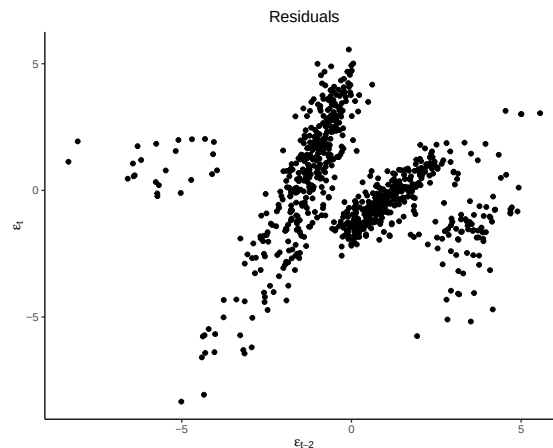


Figure 11: The 2-step residuals versus the residuals plot

6 Appendix A

The code used for solving each part is given here:

Part 1

```
rm(list = ls())
set.seed(1)

#####
#Part1
#####

#-----
# SETAR(2,1,1)
#-----

## Function to simulate SETAR model
SETAR <- function(n, a10, a11, a20, a21, th) {
  set.seed(1)
  r <- rnorm(n) # Generate errors
  y <- rep(NA, n) # Initialize the time series
  y[1] <- r[1] # Set the first value

  # Simulate the SETAR process and track regimes
  regime_colors <- character(n) # To hold colors for each point
  regime_colors[1] <- "blue" # First point color

  for (t in 2:n) {
    if (y[t-1] <= th) {
      y[t] <- a10 + a11 * y[t-1] + r[t] # First regime
      regime_colors[t] <- "blue" # Color for regime 1
    } else {
      y[t] <- a20 + a21 * y[t-1] + r[t] # Second regime
      regime_colors[t] <- "red" # Color for regime 2
    }
  }

  # Prepare the data frame for output
  D1 <- data.frame(y = y[-1], y1 = y[-n])

  # Initialize the plot with appropriate limits
  plot(D1$y, type = "n", xlab = "Time", ylab = "Value",
       main = "SETAR(2,1,1)", xlim = c(1, n),
       ylim = range(D1$y, na.rm = TRUE))

  # Draw the time series with colors
  for (t in 2:n) {
    lines(c(t-1, t), c(y[t-1], y[t]), col = regime_colors[t]) # Draw line segment
  }
```

```
# Add points to the plot for each value of y with regime-based colors
points(2:n, y[2:n], col = regime_colors[2:n], pch = 19) # 'pch = 19' for solid circle
points

# Add a dashed line at the threshold value th
abline(h = th, lty = 2, col = "black") # 'lty = 2' for dashed line, 'col = black' for color

return(D1)
}

#Plot with different parameters
par(mfrow = c(3, 1))
SETAR(n = 200, a10 = 0, a11 = 0.2, a20 = 0, a21 = -0.2, th = 0)
title(sub="a_10=0, a_11=0.5, a_20=0, a_21=-0.5")

SETAR(n = 200, a10 = 3, a11 = 0.8, a20 = -3, a21 = -0.8, th = 0)
title(sub="a_10=3, a_11=0.8, a_20=-3, a_21=-0.8")

SETAR(n = 200, a10 = -1, a11 = 0.2, a20 = 1, a21 = -0.8, th = 0)
title(sub="a_10=-1, a_11=0.3, a_20=1, a_21=-0.8")

#-----
# IGAR(2,1)
#-----

IGAR<-function(n, p, a10, a11, a20, a21){
  set.seed(1)
  ## Number of samplepoints
  ## Errors
  r <- rnorm(n)
  ## Make a time series y with a regime model
  y <- rep(NA,n)
  y[1] <- r[1]

  # Simulate the SETAR process and track regimes
  regime_colors <- character(n) # To hold colors for each point
  regime_colors[1] <- "blue" # First point color

  for(t in 2:n)
  {
    rv <- runif(1)
    if(rv < p)
    {
      y[t] <- a10 + a11 * y[t-1] + r[t]
      regime_colors[t] <- "blue" # Color for regime 1
    }
    else
    {
      y[t] <- a20 + a21 * y[t-1] + r[t]
    }
  }
}
```

```
    regime_colors[t] <- "red" # Color for regime 2
  }
}

## Put it into a data.frame, and make x1 and y1 which are lagged one step
D2 <- data.frame(y=y[-1], y1=y[-n])

# Initialize the plot with appropriate limits
plot(D2$y, type = "n", xlab = "Time", ylab = "Value",
      main = "IGAR(2,1)", xlim = c(1, n),
      ylim = range(D2$y, na.rm = TRUE))

# Draw the time series with colors
for (t in 2:n) {
  lines(c(t-1, t), c(y[t-1], y[t]), col = regime_colors[t]) # Draw line segment
}

# Add points to the plot for each value of y with regime-based colors
points(2:n, y[2:n], col = regime_colors[2:n], pch = 19) # 'pch = 19' for solid circle points

return(D2)
}

#Plot with different probabilities
par(mfrow = c(3, 1))
IGAR(n = 200, p=0.8, a10 = 4, a11 = 0.5, a20 = -4, a21 = -0.5)
title(sub="p=0.8")

IGAR(n = 200, p=0.5, a10 = 4, a11 = 0.5, a20 = -4, a21 = -0.5)
title(sub="p=0.5")

IGAR(n = 200, p=0.2, a10 = 4, a11 = 0.5, a20 = -4, a21 = -0.5)
title(sub="p=0.2")
#These results might be obvious, but the higher probability we give the observations of
  ending in regime 1
#Thus more observations land in regime 1 and vice versa.

#-----
# MMAR(2,1)
#-----

MMAR <- function(n, p, a10, a11, a20, a21){
  set.seed(1)
  ## Errors
  r <- rnorm(n)
  ## Make a time series y with a regime model
  y <- rep(NA,n)
  y[1] <- r[1]
```

```
# Simulate the SETAR process and track regimes
regime_colors <- character(n) # To hold colors for each point
regime_colors[1] <- "blue" # First point color

# Deciding if first falls into 1st or 2nd regime
rv <- runif(1)
if (rv > 0.5){
  y[2] <- a10 + a11 * y[1] + r[2]
  regime_colors[2] <- "blue" # second point color
} else {
  y[2] <- a20 + a21 * y[1] + r[2]
  regime_colors[2] <- "red" # second point color
}

for(t in 3:n)
{
  # generate rv
  rv <- runif(1)

  # If previous observation was in 1st regime
  if(y[t-1] == a10 + a11 * y[t-2] + r[t-1]){

    # select 1st regime with p=0.95
    if(rv < p)
    {
      y[t] <- a10 + a11 * y[t-1] + r[t]
      regime_colors[t] <- "blue" # Color for regime 1
    }
    # select 2nd regime with p = 0.05
    else
    {
      y[t] <- a20 + a21 * y[t-1] + r[t]
      regime_colors[t] <- "red" # Color for regime 2
    }
  }
  # else select 2nd regime with p = 0.95
  else {
    if(rv < 1-p)
    {
      y[t] <- a10 + a11 * y[t-1] + r[t]
      regime_colors[t] <- "blue" # Color for regime 1
    }
    else
    {
      y[t] <- a20 + a21 * y[t-1] + r[t]
      regime_colors[t] <- "red" # Color for regime 2
    }
  }
}
```

```
## Put it into a data.frame, and make x1 and y1 which are lagged one step
D3 <- data.frame(y=y[-1], y1=y[-n])

# Initialize the plot with appropriate limits
plot(D3$y, type = "n", xlab = "Time", ylab = "Value",
      main = "MMAR(2,1)", xlim = c(1, n),
      ylim = range(D3$y, na.rm = TRUE))

# Draw the time series with colors
for (t in 2:n) {
  lines(c(t-1, t), c(y[t-1], y[t]), col = regime_colors[t]) # Draw line segment
}

# Add points to the plot for each value of y with regime-based colors
points(2:n, y[2:n], col = regime_colors[2:n], pch = 19) # 'pch = 19' for solid circle points

return(D3)
}

#Plot with different transition probabilities
par(mfrow = c(3, 1))
MMAR(n=200, p=0.95, a10=4, a11=1, a20=-10, a21=-0.5)
title(sub="p1=p2=0.95")

MMAR(n=200, p=0.5, a10=4, a11=1, a20=-10, a21=-0.5)
title(sub="p1=p2=0.5")

MMAR(n=200, p=0.25, a10=4, a11=1, a20=-10, a21=-0.5)
title(sub="p1=p2=0.25")

#Compare all three models with same parameters
par(mfrow = c(3, 1))
SETAR(n=200, a10=0, a11=0.7, a20=0, a21=-0.7, th=0)
IGAR(n=200, p=0.7, a10=0, a11=0.7, a20=0, a21=-0.7)
MMAR(n=200, p=0.7, a10=0, a11=0.7, a20=0, a21=-0.7)
```

Part 2

```
# Part 2
#####

rm(list = ls())
dev.off()
set.seed(1)

# Simulate values from a SETAR model -----

n <- 1000 # Number of sample points
r <- rnorm(n) # Errors

# Make a time series y with a regime model
y <- rep(NA,n)
y[1] <- r[1]

for(t in 2:n)
{
  if(y[t-1] >= 0)
  {
    y[t] <- -0.8 * y[t-1] + r[t]
  }
  else
  {
    y[t] <- 1.5 + 0.8* y[t-1] + r[t]
  }
}

# Collect into a data frame
D <- data.frame(y = y[-1], y1 = y[-n] )

# Find estimation of conditional mean -----

## Epanechnikov kernel
kernelEp <- function(xall,x,h)
{
  ## Make the weights with an Epanechnikov kernel
  ## h has the same unit as x (i.e. it is on the same absolute scale, so if x
  ## is Watt, h is also given in Watt)
  u <- abs(xall-x)
  u <- u / h
  w <- 3/(4*h) * (1 - u^2)
  ## Set values with |u|>1 to 0
  w[abs(u)>1] <- 0
  return(w)
}
```

```
# function for computing m_hat
m_hat <- function(x,h){
  1/n * sum(D$y*kernelEp(D$y1,x,h))/(1/n * sum(kernelEp(D$y1,x,h)))
}

# compute and store for plotting with different bandwidths
x <- seq(min(D$y),max(D$y),length.out = 50)
m_hat_b_2 <- numeric(length(x))
m_hat_b_5 <- numeric(length(x))
m_hat_b_8 <- numeric(length(x))
m_hat_b_15 <- numeric(length(x))

for (i in 1:length(x)){
  m_hat_b_2[i] <- m_hat(x[i], 0.2)
  m_hat_b_5[i] <- m_hat(x[i], 0.5)
  m_hat_b_8[i] <- m_hat(x[i], 0.8)
  m_hat_b_15[i] <- m_hat(x[i], 1.5)
}

# Compute theoretical conditional mean
m <- numeric(length(x))
for (i in 1:length(x)){
  if(x[i]>=0){
    m[i] <- -0.8 * x[i]
  } else {
    m[i] <- 1.5 + 0.8*x[i]
  }
}

# Data frame for plotting
df_plot <- data.frame(x = x,
                      m_hat_b_2 = m_hat_b_2,
                      m_hat_b_5 = m_hat_b_5,
                      m_hat_b_8 = m_hat_b_8,
                      m_hat_b_15 = m_hat_b_15,
                      m = m )

plot_cond_mean_epa <- ggplot(df_plot, aes(x = x)) +
  geom_point(data = D , aes(x = y1, y = y), color = "gray90") +
  geom_line(aes(y = m_hat_b_2, color = "h = 0.2")) +
  geom_line(aes(y = m_hat_b_5, color = "h = 0.5")) +
  geom_line(aes(y = m_hat_b_8, color = "h = 0.8")) +
  geom_line(aes(y = m_hat_b_15, color = "h = 1.5")) +
  geom_point(aes(y = m), shape = 3, color = "gray70") +
  scale_color_manual(values = c("h = 0.2" = "yellow4",
                                "h = 0.5" = "blue",
                                "h = 0.8" = "orange",
                                "h = 1.5" = "red")) +
  labs(title = "With Epanechnikov Kernel", y = "M(x)", x = "x", color = "Bandwidths") +
```

```
theme_classic()

#####
# Estimation of mean with loess()
#####

## Estimate the function with 2D local polynomial regression using loess()
fit1 <- loess(y ~ y1, dat=D, span=0.3)
fit2 <- loess(y ~ y1, dat=D, span=0.5)
fit3 <- loess(y ~ y1, dat=D, span=0.8)

y_fit1 <- predict(fit1, newdata = x)
y_fit2 <- predict(fit2, newdata = x)
y_fit3 <- predict(fit3, newdata = x)

# Create a data frame for the fitted values
d_fit <- data.frame(x = x,
                    true_mean = m,
                    y_fit1 = y_fit1,
                    y_fit2 = y_fit2,
                    y_fit3 = y_fit3)

# Part 4: Plot true_mean, y_fit1, y_fit2, and y_fit3 together
plot_cond_mean_Tric <- ggplot(d_fit, aes(x = x)) +
  geom_point(data = D , aes(x = y1, y = y), color = "gray90") +
  geom_point(aes(y = true_mean ), color = "gray60", shape = 3 ) +
  geom_line(aes(y = y_fit1, color = "Span = 0.3")) +
  geom_line(aes(y = y_fit2, color = "Span = 0.5")) +
  geom_line(aes(y = y_fit3, color = "Span = 0.8")) +
  labs( title = "With Tricube Kernel",
        x = "x",
        y = "M(x)") +
  theme_classic() +
  scale_color_manual(values = c(
    "Span = 0.3" = "yellow4",
    "Span = 0.5" = "blue",
    "Span = 0.8" = "red")) +
  guides(color = guide_legend(title = "Data Proportion"))

grid.arrange(plot_cond_mean_epa, plot_cond_mean_Tric, ncol = 2)
```

Part 3

```
#####  
# Part 3  
#####  
  
rm(list = ls())  
dev.off()  
set.seed(1)  
  
## Simulate for the same SETAR(2,1,1) as before -----  
  
n <- 1000  
r <- rnorm(n)  
x <- rep(NA,n)  
x[1] <- r[1]  
  
for(t in 2:n)  
{  
  if(x[t-1] >= 0)  
  {  
    x[t] <- -0.8 * x[t-1] + r[t]  
  }  
  else  
  {  
    x[t] <- 1.5 + 0.8* x[t-1] + r[t]  
  }  
}  
  
# Computing theoretical cumulative conditional mean -----  
Lambda.dot <- function(dot, a){  
  # If dot is less than the point where the function is discontinuous  
  if (dot < 0){  
    # Integrate from a until that point  
    Lambda <- (1.5*dot + 0.8/2 *dot^2) - (1.5*a + 0.8/2 *(a)^2)  
  }  
  # If dot is more than the point where the function is discontinuous, then  
  # integrate the first function from -2 until 0 and the second function  
  # from 0 until dot  
  else {  
    Lambda <- - (1.5*a + 0.8/2 *(a)^2) + ( -0.8/2*dot^2)  
  }  
  return(Lambda)  
}  
  
x_theo <- seq(-4, 4, 0.1)  
Lambda.dot_storage <- numeric(length(x_theo))  
for (i in 1:length(x_theo)){  
  Lambda.dot_storage[i] <- Lambda.dot(x_theo[i], -4)
```

```
}

plot(Lambda.dot_storage)

# Finding the estimate for Cumulative conditional mean -----

## Number of intervals
n.bin <- 20
## The breaks between the intervals
breaks <- seq(-3.3,3.3,len=n.bin+1)
## Initialize
h <- diff(breaks)[1]
lambda <- gamma <- f.hat <- h.hat <- numeric(n.bin)

## Cut into intervals conditioned on x_{t-1}
# The first element of L will contain the values of x_{t} where x_{t-1} are contained
# in the first interval e.g. [-3,-1.8]
L <- split(x[-1], cut(x[-length(x)],breaks))
## Check if there are at least 5 points in each interval
if(!all(sapply(L,length)>=5)){ print('Stopped: There are less than 5 points in one of the
  intervals'); break;}
## Calc the hist regressogram, i.e. for each interval
for(i in 1:n.bin)
{
  x.bin <- L[[i]]
  lambda[i] <- mean(x.bin)
  f.hat[i] <- (n.bin*h)^(-1) * length(x.bin)
  gamma[i] <- sum((x.bin - lambda[i])^2) / length(x.bin)
}

# The the estimate for Cumulative conditional mean
Lambda_esti <- cumsum(lambda*h)
dot_esti <- (breaks[-length(breaks)] + breaks[-1]) / 2 #breaks[-1] #

## 95% confidence band, c is found in table 3.1
c.alpha <- 1.273
for(i in 1:n.bin)
{
  h.hat[i] <- gamma[i]/f.hat[i];
}
H.hat <- cumsum(h.hat*h);
##
H.hat.b <- H.hat[n.bin];
Lambda.lower <- Lambda_esti - c.alpha * n.bin^(-0.5) * H.hat.b^(0.5) * (1 + H.hat/H.hat.b);
Lambda.upper <- Lambda_esti + c.alpha * n.bin^(-0.5) * H.hat.b^(0.5) * (1 + H.hat/H.hat.b);

plot(Lambda_esti)
```

```
par(mfrow=c(1, 2))

df_esti <- data.frame(dot_esti, Lambda_esti, Lambda.lower, Lambda.upper)
df_true <- data.frame(x_theo, Lambda.dot_storage)

ggplot() +
  geom_line(data = df_esti, aes(x = dot_esti, y = Lambda_esti, color = "Estimated"),
    linetype = "solid") +
  geom_line(data = df_esti, aes(x = dot_esti, y = Lambda.upper, color = "CI"), linetype =
    "dashed") +
  geom_line(data = df_esti, aes(x = dot_esti, y = Lambda.lower ), color = "red", linetype =
    "dashed") +
  geom_line(data = df_true, aes(x = x_theo, y = Lambda.dot_storage , color = "Theoretical"),
    linetype = "solid") +
  labs( y = "Cumulative Conditional Mean", x = "") +
  theme_minimal() +
  scale_color_manual(values = c(
    "Estimated" = "black",
    "CI" = "red",
    "Theoretical" = "blue"))+
  guides(color = guide_legend(title = ""))
```

Part 4

```
#####  
# Part 4  
#####  
  
rm(list = ls())  
dev.off()  
df <- read.csv("DataPart4.csv")  
df$Ua <- df$Ph/(df$Ti - df$Te)  
  
kernelEp <- function(xall,x,h)  
{  
  ## Make the weights with an Epanechnikov kernel  
  ## h has the same unit as x (i.e. it is on the same absolute scale, so if x is Watt, h is  
  ## also given in Watt)  
  u <- abs(xall-x)  
  u <- u / h  
  w <- 3/4 * (1 - u^2)  
  ## Set values with |u|>1 to 0  
  w[abs(u)>1] <- 0  
  return(w)  
}  
  
wind_speed <- seq(min(df$W), max(df$W), 0.01)  
  
## quadratic locally-weighted -----  
  
# storage for different bandwidths  
Ua_hat_08_quad <- numeric(length(wind_speed))  
Ua_hat_1_quad <- numeric(length(wind_speed))  
Ua_hat_15_quad <- numeric(length(wind_speed))  
  
# Bandwidth 0.8  
for(i in 1:length(wind_speed)){  
  
  w <- kernelEp(df$W, wind_speed[i], 0.8)  
  ok <- w>0  
  model <- lm(Ua ~ W + I(W^2) , data = df[ok,], weights = w[ok])  
  Ua_hat_08_quad[i] <- as.numeric(coefficients(model)[1]) +  
    as.numeric(coefficients(model)[2])*wind_speed[i] +  
    as.numeric(coefficients(model)[3])*wind_speed[i]^2  
}  
  
# Bandwidth 1  
for(i in 1:length(wind_speed)){  
  
  w <- kernelEp(df$W, wind_speed[i], 1)  
  ok <- w>0
```

```
model <- lm(Ua ~ W + I(W^2) , data = df[ok,], weights = w[ok])
Ua_hat_1_quad[i] <- as.numeric(coefficients(model)[1]) +
  as.numeric(coefficients(model)[2])*wind_speed[i] +
  as.numeric(coefficients(model)[3])*wind_speed[i]^2
}

# Bandwidth 1.5
for(i in 1:length(wind_speed)){

  w <- kernelEp(df$W, wind_speed[i], 1.5)
  ok <- w>0
  model <- lm(Ua ~ W + I(W^2) , data = df[ok,], weights = w[ok])
  Ua_hat_15_quad[i] <- as.numeric(coefficients(model)[1]) +
    as.numeric(coefficients(model)[2])*wind_speed[i] +
    as.numeric(coefficients(model)[3])*wind_speed[i]^2
}

## plot -----
df <- data.frame(wind_speed, Ua_hat_08_quad, Ua_hat_1_quad, Ua_hat_15_quad)

ggplot() +
  geom_line(data = df, aes(x = wind_speed, y = Ua_hat_08_quad, color = "h = 0.8"), linetype
    = "solid") +
  geom_line(data = df, aes(x = wind_speed, y = Ua_hat_1_quad, color = "h = 1" ), linetype =
    "solid") +
  geom_line(data = df, aes(x = wind_speed, y = Ua_hat_15_quad , color = "h = 1.5"), linetype
    = "solid") +
  labs( y = "Heat Loss Coefficient", x = "Wind Speed") +
  theme_minimal() +
  scale_color_manual(values = c(
    "h = 0.8" = "green4",
    "h = 1" = "red",
    "h = 1.5" = "blue"))+
  guides(color = guide_legend(title = ""))
```

Part 5

```
rm(list = ls())

# Load necessary libraries
library(tseries)      # For ARMA function
library(forecast)     # For ARIMA and diagnostics
library(ggplot2)      # For visualizations
library(stats)        # For ts diagnostics

# Load custom functions for diagnostics (assuming my.tsdiag is defined in your environment)
source('tsa_functions.R') # Assuming this file contains tsa.plot and my.tsdiag

# Load the dataset
data <- read.csv("DataPart5.csv")
series <- data$x

# Plot ACF and PACF to start with initial diagnostics
par(mfrow=c(1,2)) # Set up 1x2 plotting area
acf(data$x, main="ACF of Original Data")
pacf(data$x, main="PACF of Original Data")

#Fit an ARMA(1,2)
arma12 <- arima(data$x, order=c(1, 0, 2))
my.tsdiag(arma12)
Box.test(residuals(arma12), lag=10, type="Ljung-Box")

#Try to fit an initial ARMA(1,1) model
arma11 <- arima(data$x, order=c(1, 0, 1))
summary(arma11)

# Use my.tsdiag to check residuals of the ARMA(1,1) model
my.tsdiag(arma11)

# Check Ljung-Box test
Box.test(residuals(arma11), lag=10, type="Ljung-Box")

#####
# Estimating LDF
#####
arma12_res <- residuals(arma12)

source("leaveOneOut.R")
source("ldf.R")

# Set the lags for which you want to calculate the LDF
lags <- 1:12

# Calculate and plot the LDF
```

```
ldf_values <- ldf(arma12_res, lags, nBoot = 30, plotIt = TRUE, plotFits = FALSE)
ldf_values
```

```
# Create residual plot
df_residuals <- data.frame(arma12_res,
                           r_2 = c(0,0,arma12_res[c(-799,-800)]),
)
```

```
res_plot <- ggplot(data = df_residuals, aes(x = r_2, y = arma12_res)) +
  geom_point() +
  labs(x = expression(epsilon[t-2]), y = expression(epsilon[t]), title = "Residuals") +
  theme_classic() +
  theme(plot.title = element_text(hjust = 0.5))
```

```
res_plot
```
